

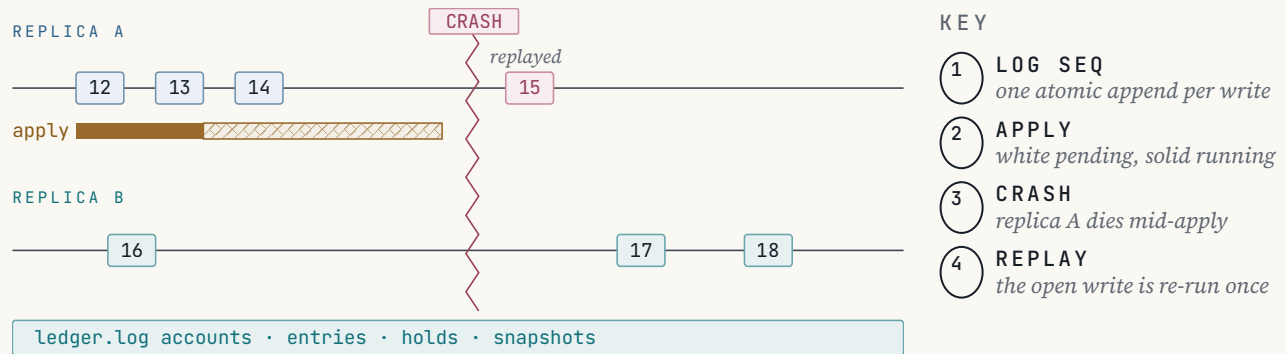


FIG. 2 • HOLDS



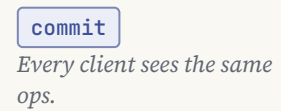
FIG. 3 • SNAPSHOTS



FIG. 4 • REPLAY



FIG. 5 • WATCHERS



# Ledger Service

## Technical Manual

The double-entry ledger from the first write down to the stored rows: atomic appends, holds that settle exactly once, replay after a crash, snapshots, and the storage underneath.

# Contents

## 1 The Model

|     |                              |   |
|-----|------------------------------|---|
| 1.1 | What the ledger is for ..... | 4 |
| 1.2 | The life of one write .....  | 6 |

## 2 Operations

|     |                                |   |
|-----|--------------------------------|---|
| 2.1 | Running it in production ..... | 9 |
|-----|--------------------------------|---|

# The Model

*A ledger whose every visible balance is a stored append, so a process that dies mid-write is just a pause.*

---

|     |                        |   |
|-----|------------------------|---|
| 1.1 | What the ledger is for | 4 |
| 1.2 | The life of one write  | 6 |

---

01

# What the ledger is for

*Money moves in pairs. The ledger stores both halves in one append, and nothing is shown that was not appended.*

---

This manual documents `ledger-svc` at version 2.4.0, from repository `acme/ledger` at commit `5b6c792`. It follows the service from its one rule down to the rows on disk, and every record shown was captured from a real run `NOTES.md §2`.

## The problem: four kinds of state

A payment touches more than a balance. Before the ledger, each of these lived somewhere different:

---

**Entries** — the debit and credit lines, appended in pairs [spec §3](#).

---

**Holds** — reserved funds that settle or release exactly once. See [the life of one write \(p. 6\)](#).

---

**Snapshots** — periodic balances so readers never scan the whole log.

---

### EXPERIMENTAL

The README opens with one line of status: “The API changes without notice between releases.” Everything here is true of 2.4.0.

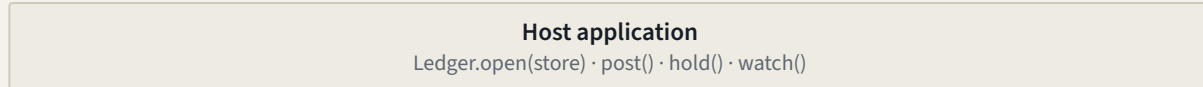
### CORE RULE

“A write atomically appends both entries and any hold change. Only appended state is observable.”

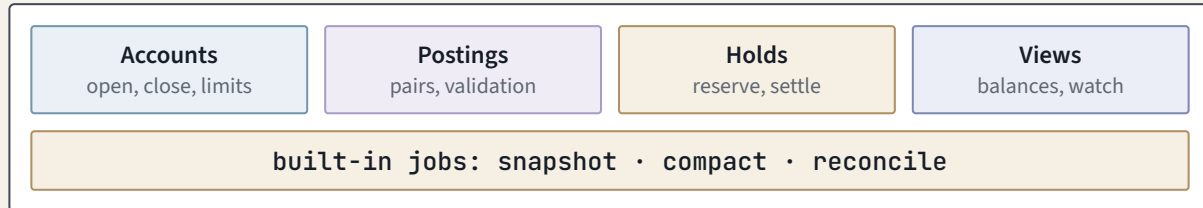
FIG. 1.1 THE LAYERS OF THE SERVICE

LAYERS

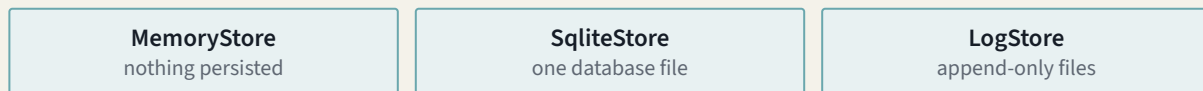
## YOUR PROCESS



## LEDGER · SRC/LEDGER



## STORAGE · ONE BACKEND PER LEDGER



The host drives the ledger, and the ledger appends through one store. The right column of options is omitted. After src/ledger/index.ts.

## How the sources rank

| SOURCE      | PATH           | USED FOR                                                    |
|-------------|----------------|-------------------------------------------------------------|
| Source code | src/**         | every type, default and field name                          |
| Spec        | docs/spec.md   | every rule, quoted exactly; cited as <code>spec §5.2</code> |
| Tests       | test/*.test.ts | exact behaviour at the edges                                |

## WHAT THIS MEANS FOR YOU

- Ask one question of any design: is the balance you show appended? If not, it does not exist yet.
- Change related things in one write, so a crash can never split them.
- Never call the network inside a write callback.

Sources: docs/spec.md (§1, §3); src/ledger/index.ts; test/holds.test.ts ("settles once"); NOTES.md §2

## 1.2

# The life of one write

*One payment, eight appends, and the order in which a reader can see them.*

A **write** is an atomic append: everything in it is stored together, or nothing is. Here is the smallest program that posts one payment:

Quick start, minimal examples/00-post.ts

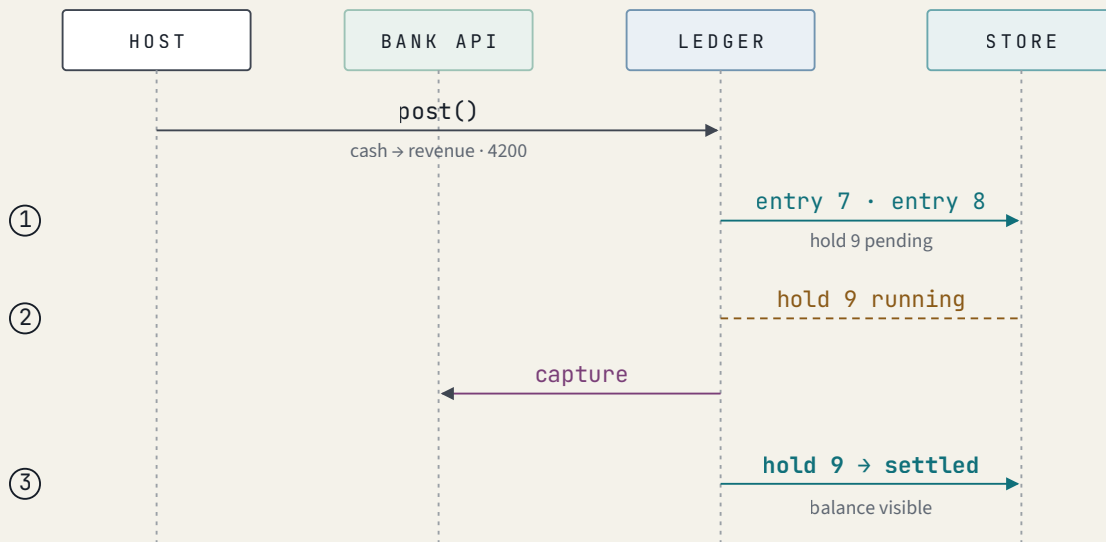
TS

```
const ledger = await Ledger.open(new SqliteStore("ledger.db"));
await ledger.post({
  debit: "cash",
  credit: "revenue",
  amount: 4200, // cents
});
```

Run with `npm run tsx examples/00-post.ts`; the database is created on first open.

FIG. 1.2 ONE PAYMENT, APPENDS 1-4

SEQUENCE



**Appends 1-3 take the payment from `post()` to a settled hold.** From [research/capture/sqlite-commits.txt](#).

## Appends 2-3: settling the hold

The scheduler reserves hold 9 by rewriting it from `pending` to `running`, then calls the bank off the line. A crash between the two leaves an open hold that is replayed once on restart running it in production (p. 9).

**FIRE AND FORGET**

Work started inside the callback and left running can still change the open write, and then fails once it settles. Await every call.

---

Sources: `src/ledger/post.ts`; `src/holds/scheduler.ts`; `research/capture/sqlite-commits.txt`

# Operations

*What to watch, what to page on, and what the numbers looked like last quarter.*

---

## 2.1 Running it in production

9

---

02

## 2 . 1

# Running it in production

*Four numbers tell you whether the ledger is healthy; one tells you it is not.*

|                                               |                                            |                                           |                                            |
|-----------------------------------------------|--------------------------------------------|-------------------------------------------|--------------------------------------------|
| <b>99.98%</b><br><small>WRITE SUCCESS</small> | <b>4.1 ms</b><br><small>P50 APPEND</small> | <b>38 ms</b><br><small>P99 APPEND</small> | <b>0</b><br><small>UNSETTLED HOLDS</small> |
|-----------------------------------------------|--------------------------------------------|-------------------------------------------|--------------------------------------------|

## Settings and defaults

| FIELD         | TYPE        | DEFAULT  | SEE                        |
|---------------|-------------|----------|----------------------------|
| store         | Store       | required | <a href="#">1.1 (p. 4)</a> |
| snapshotEvery | number      | 1000     | <a href="#">1.2 (p. 6)</a> |
| now           | () ⇒ number | Date.now | —                          |

### TIP

Alert on unsettled holds older than five minutes, not on error rate: a stuck hold is the first sign of a poisoned store.

### WHAT THIS MEANS FOR YOU

- Page on stuck holds; dashboard everything else.
- If storage fails in a way that may have written, close and reopen; do not retry.

Sources: ops/runbook.md; dashboards/ledger.json (Q3 2026)